

Onyx — Changes applied to Circle

Onyx · multi-process OS on Raspberry Pi 4

Onyx uses **Circle** ([rsta2/circle](https://github.com/rsta2/circle)) as its hardware-abstraction + driver layer, kept as close to upstream as possible. The handful of changes we need live in a **fork**, pulled in as a **git submodule**:

- **Submodule:** `circle/` → <https://github.com/stephaneweg/circle.git> (`.gitmodules`), branch **onyx**.
- **Upstream base:** <https://github.com/rsta2/circle.git>, tag **Step51** (remote upstream).
- The superproject pins a specific fork commit; `git submodule status` shows it (currently `Step51-6-g...`).

The patches on top of `Step51` are small and surgical. The diffs below are the exact output of `git diff Step51..onyx` inside `circle/`. To reproduce:

```
git -C circle remote -v # origin = the fork, upstream = rsta2/circle
git -C circle log --oneline Step51..onyx
git -C circle diff Step51..onyx
```

Design rule. Patch Circle as little as possible. When a change can be header-only (no library rebuild), it is. Anything that can live in our own kernel instead (for example the cooked-mouse changed-button handling, done in `kernel/`'s mouse stub) is **not** a Circle patch.

Some in-source comments still say "Zircon" (the project's former codename) instead of "Onyx" — cosmetic only.

Summary

#	Patch	Files touched	Rebuild
1	Keyboard map decoupled from the kernel (no compiled-in country maps; loaded at runtime) · <code>MAX_TASKS 20</code> → <code>40</code>	<code>input/keymap.{h,cpp}</code> (+ 7 <code>keymap_*.h</code> deleted), <code>usb/usbkeyboard.h</code> , <code>input/keyboardbehaviour.h</code> , <code>sysconfig.h</code>	<code>libinput</code> (+ kernel for <code>MAX_TASKS</code>)
2	Heap large-block reuse (fix per-launch canvas leak) + free-list byte accounting	<code>heapallocator.{h,cpp}</code> , <code>sysconfig.h</code>	<code>libcircle</code>
3	Free-space accounting in the page	<code>memory.h</code> , <code>pageallocator.{h,cpp}</code>	<code>libcircle</code> (<code>memory.h</code> header-only)

#	Patch	Files touched	Rebuild
	allocator + memory accessors (for the <code>meminfo</code> kapi)		
4	Multi-core enabled (<code>ARM_ALLOW_MULT_I_CORE</code>): core 1 runs the sound producer	<code>sysconfig.h</code>	every library (clean rebuild) + kernel
5	Shift + navigation keys (<code>KeyShiftUp...</code> appended to <code>TSpecialKey</code> , <code>xterm ;2</code> sequences)	<code>input/keymap.{h,cpp}</code>	<code>libinput</code> , <code>libusb</code> + the <code>.kmap</code> files
6	Partial display update <code>C2DGraphics::UpdateDisplay(x, y, w, h)</code> (the compositor's dirty rectangles)	<code>2dgraphics.{h,cpp}</code>	<code>libcircle</code>
7	Yielding SD waits: weak hooks for the FatFs volume lock and the EMMC wait loop	<code>addon/fatfs/ffsystem.cpp</code> , <code>addon/SDCard/emmc.cpp</code>	<code>libfatfs</code> , <code>libsdcard</code>
8	Sector cache in the FatFs disk layer (write-through), one bounce buffer per volume	<code>addon/fatfs/diskio.{h,cpp}</code>	<code>libfatfs</code>
10	Multi-cluster transfers in <code>f_read / f_write</code> (one SD command across contiguous clusters)	<code>addon/fatfs/ff.c</code>	<code>libfatfs</code>
9	High Speed SD at run time (<code>CEMMCDevice::SetHighSpeed</code>) + the host's High	<code>addon/SDCard/emmc.{h,cpp}</code>	<code>libsdcard</code>

#	Patch	Files touched	Rebuild
	Speed Enable bit		
11	FatFs fast seek on (FF_USE_FASTSEEK 1: kapi_seek in a GameCube disc image)	addon/fatfs/ffconf.h	libfatfs + the kernel (the FILE layout: clean rebuild)
14	DHCP restart (joining another Wi-Fi network while running; the reconfiguration itself: the kernel's link-time wrap, no hostap change)	include/circle/net/dhcpclient.h, lib/net/dhcpclient.cpp	libnet (+ the kernel)
13	The SD card's partitions as volumes SD: SD1: ... SD3: + exFAT on	addon/fatfs/ffconf.h, addon/fatfs/diskio.cpp, addon/fatfs/ff.c	libfatfs + the kernel (FSIZE_t is 64-bit: clean rebuild)
17	The host name set after the constructor (CNetSubSystem::SetHostname: system.ini's hostname=)	include/circle/net/netsubsystem.h	none (inline) — the kernel
7b	Yielding SD waits, many short ones: a second weak hook OnyxDriverPoll at each turn of a short wait	addon/SDCard/emmc.cpp	libsdcard
20	TCP: real duplicate ACKs only (RFC 5681 §2) — the peer's data segments no longer start a fast retransmit	lib/net/tcpconnection.cpp	libnet
21	TCP: RTO 1 s initial (was 3 s; the 1 s minimum kept)	lib/net/retranstimeoutcalc.cpp, include/circle/net/retranstime	libnet

#	Patch	Files touched	Rebuild
	— 200 ms was tried and undone), Karn after a fast retransmit	outcalc.h, lib/net/tcpconnection.cpp	
22	CSocket::Send's count: the bytes queued when a later chunk times out	lib/net/socket.cpp	libnet
12	2D DMA with a source stride (a rectangle read in place: no gathering) + an asynchronous partial update (the compositor yields instead of spinning)	dmachannel. {h,cpp}, dma4channel. {h,cpp}, bcmframebuffer. {h,cpp}, 2dgraphics. {h,cpp}	libcircle

1. Keyboard map decoupled from the kernel (+ MAX_TASKS)

Why. Circle's cooked-mode keyboard bakes the country maps into the binary (one selected at boot via `keymap=`) and offers no way to change it afterwards. Onyx wants **live** layout switching *and* layouts that ship as **data files** (`SD:/etc/keymaps/*.kmap`), so a new layout needs no kernel rebuild — and the kernel carries **no** keyboard tables at all.

What.

1. **Expose the keyboard's CKeyMap** up the device stack (`keyboardbehaviour.h`, `usbkeyboard.h` gain a `GetKeyMap()`), so the kernel can reach the live map and fill it through the existing public `CKeyMap::SetEntry / ClearTable`.
2. **Drop the compiled-in maps.** `keymap.cpp` no longer `#includes` the `keymap_*.h` tables, and `CKeyMap` loses `s_DefaultMap[]`, `s_MapDirectory[]` and `LookupDefaultMap()`; the constructor just zeroes the map (every entry `KeyNone`). The 7 stock tables (`keymap_{de,dv,es,fr,it,uk,us}.h`, **905 lines**) are **deleted** from the Circle tree — they now live in the Onyx repo at `tools/keymaps/maps/` and are compiled to `.kmap` blobs by `tools/keymaps/genkeymaps.py`.

The kernel fills the live keyboard from a `.kmap` file via `SetEntry` (`kapi_set_keymap_data`, ABI v27), re-applying it whenever a keyboard (re-)attaches. `MAX_TASKS` goes **20 → 40**: the network stack adds long-lived background tasks (net / DHCP / WPA supplicant + NTP + IRC).

```

--- a/include/circle/input/keymap.h
+++ b/include/circle/input/keymap.h
@@ -130,8 +130,9 @@ public:
    u8 GetLEDStatus (void) const;

-private:
-    static const void *LookupDefaultMap (const char *pLocale);

```

```

+ // Zircon: the keyboard map is empty until the kernel fills it at runtime via
+ // ClearTable/SetEntry from a SD:/etc/keymaps/*.kmap file -- no country map is
+ // compiled in (see CKeyMap::CKeyMap and kapi_set_keymap_data).

private:
    u16 m_KeyMap[PHY_MAX_CODE+1][K_CTRLTAB+1];
@@ -141,8 +142,6 @@ private:
    boolean m_bScrollLock;

    static const char *s_KeyStrings[KeyMaxCode-KeySpace];
-   static const u16 s_DefaultMap[][PHY_MAX_CODE+1][K_CTRLTAB+1];
-   static const char *s_MapDirectory[];
};

--- a/lib/input/keymap.cpp
+++ b/lib/input/keymap.cpp
@@ -96,60 +96,16 @@ const char *CKeyMap::s_KeyStrings[KeyMaxCode-KeySpace] =
    "." // KeyKP_Period
};

-#define C(chr) ((u16) (u8) (chr))
-
-const u16 CKeyMap::s_DefaultMap[][PHY_MAX_CODE+1][K_CTRLTAB+1] =
-{-
-    {
-        #include "keymap_de.h"
-    }, {
-        #include "keymap_dv.h"
-    }, {
-        ... (fr, es, it, uk, us)
-    }
-};
-
-const char *CKeyMap::s_MapDirectory[] = { "DE", "DV", "ES", "FR", "IT", "UK", "US", 0 };
-
+// Zircon: the kernel compiles in NO country maps -- keyboard layouts ship as
+// SD:/etc/keymaps/*.kmap data files and are loaded at runtime through
+ClearTable/SetEntry
+// (see kapi_set_keymap_data). So a fresh keymap starts empty (every entry KeyNone).
CKeyMap::CKeyMap (void)
:   m_bCapsLock (FALSE), m_bNumLock (FALSE), m_bScrollLock (FALSE)
{
-   const char *pLocale = CKernelOptions::Get ()->GetKeyMap ();
-   const void *pDefaultMap = LookupDefaultMap (pLocale);
-   ... (fallback to DEFAULT_KEYMAP)
-   memcpy (m_KeyMap, pDefaultMap, sizeof m_KeyMap);
+   memset (m_KeyMap, 0, sizeof m_KeyMap); // KeyNone == 0
}

@@ -336,17 +292,3 @@ u8 CKeyMap::GetLEDStatus (void) const
-const void *CKeyMap::LookupDefaultMap (const char *pLocale)
-{-
-   for (unsigned nMap = 0; s_MapDirectory[nMap] != 0; nMap++)
-       if (strcmp (s_MapDirectory[nMap], pLocale) == 0)
-           return s_DefaultMap[nMap];
-   return 0;
-}

--- a/include/circle/input/keyboardbehaviour.h
+++ b/include/circle/input/keyboardbehaviour.h
@@ -69,6 +69,8 @@ public:
    u8 GetLEDStatus (void) const;

```

```

+ CKeyMap *GetKeyMap (void) { return &m_KeyMap; } // Zircon: runtime layout
switch
+
private:
    void GenerateKeyEvent (u8 ucKeyCode);

--- a/include/circle/usb/usbkeyboard.h
+++ b/include/circle/usb/usbkeyboard.h
@@ -62,6 +62,9 @@ public:
    boolean SetLEDs (u8 ucStatus);

+ // Zircon: access the cooked-mode key map so the layout can be switched at
runtime.
+ CKeyMap *GetKeyMap (void) { return m_Behaviour.GetKeyMap (); }
+
private:
    void ReportHandler (const u8 *pReport, unsigned nReportSize);

--- a/include/circle/sysconfig.h
+++ b/include/circle/sysconfig.h
@@ -273,7 +277,8 @@
    #ifndef MAX_TASKS
    -#define MAX_TASKS      20
+ #define MAX_TASKS      40 // Onyx: bumped from 20 -- net stack adds
+ // background tasks (net/DHCP/WPA + NTP/IRC)
    #endif

```

(The 7 lib/input/keymap_*.h tables are deleted — 905 lines — and not shown here.)

2. Heap large-block reuse (fix per-launch leak)

Why. Circle's CHeapAllocator keeps freed blocks on per-size **bucket** free lists, but the buckets top out at $0 \times 80000 = 512$ KB. A block **bigger than the top bucket cannot be returned to any free list and is lost** on Free(). Onyx allocates large blocks routinely — a window **canvas** is up to ~ 3 MB ($1024 \times 768 \times 4$) — so **every app launch leaked its canvas**.

Fix. Add **power-of-two "large" free lists** alongside the buckets: a too-big block is rounded to its power-of-two class and pushed onto m_pLargeFreeList[exp] on free, and reused from it on allocate. A running m_nFreeListBytes counter (exposed via GetFreeListSpace()) feeds the meminfo accounting (patch 3). **Rebuild libcircle** after this change.

```

--- a/include/circle/heapallocator.h
+++ b/include/circle/heapallocator.h
@@ -40,6 +40,12 @@ ASSERT_STATIC (DATA_CACHE_LINE_LENGTH_MAX >= 16);
    #define HEAP_BLOCK_MAX_BUCKETS 20

+// Onyx: blocks larger than the biggest bucket are rounded up to a power of two and
+// kept on a per-power free list (index = log2 of the rounded size), so large blocks
+// (window canvases, big file buffers, ...) are reused instead of lost on free. 32
+// classes cover any realistic size.
+ #define HEAP_LARGE_LISTS      32
+
+ struct THeapBlockHeader
@@ -83,6 +89,10 @@ public:
    size_t GetFreeSpace (void) const;

+ /// \return Onyx: bytes of freed blocks currently on the bucket/large free lists

```

```

+   /// (reusable). Add to GetFreeSpace() for the true total free space.
+   size_t GetFreeListSpace (void) const { return m_nFreeListBytes; }
+
+   /// \param nSize Block size to be allocated
@@ -95,8 +105,8 @@ public:
+   /// \param pBlock Memory block to be freed
-   /// \note Memory space of blocks, which are bigger than the largest bucket size,
-   /// cannot be returned to a free list and is lost.
+   /// \note Onyx: blocks bigger than the largest bucket are rounded to a power of
two
+   /// and returned to a per-power free list (reused), so they are NOT lost.
+   void Free (void *pBlock);
@@ -117,6 +127,8 @@ private:
+   THeapBlockBucket m_Bucket[HEAP_BLOCK_MAX_BUCKETS+1];
+   THeapBlockHeader *m_pLargeFreeList[HEAP_LARGE_LISTS]; // per-power-of-2 (Onyx)
+   size_t m_nFreeListBytes; // Onyx: bytes on the bucket+large free lists
+   CSpinLock m_SpinLock;

--- a/lib/heapallocator.cpp
+++ b/lib/heapallocator.cpp
@@ -53,6 +53,45 @@ void CHeapAllocator::Setup (uintptr nBase, size_t nSize, size_t
nReserve)
+   m_nReserve = nReserve;
+
+   for (unsigned i = 0; i < HEAP_LARGE_LISTS; i++) // Onyx: large-block free
lists
+       m_pLargeFreeList[i] = 0;
+   m_nFreeListBytes = 0;
+}
+
+// Onyx: round a too-big request up to a power-of-two class; return its free-list
index.
+static int HeapLargeClass (size_t *pnSize)
+{
+   size_t nRounded = HEAP_BLOCK_ALIGN; int nExp = 0;
+   while (nRounded < *pnSize) { nRounded <= 1; if (++nExp >= HEAP_LARGE_LISTS)
return -1; }
+   *pnSize = nRounded; return nExp;
+}
+// Free-list index of an already-rounded large block (stored size = ALIGN << exp).
+static int HeapLargeIndex (size_t nRoundedSize)
+{
+   int nExp = 0;
+   while ((size_t) HEAP_BLOCK_ALIGN << nExp) < nRoundedSize)
+       if (++nExp >= HEAP_LARGE_LISTS) return -1;
+   return nExp;
+}
@@ -96,12 +135,28 @@ void *CHeapAllocator::DoAllocate (size_t nSize)
+   // Onyx: too big for every bucket -> round to a power-of-two class so it can be
+   // reused from m_pLargeFreeList instead of being lost on free.
+   int nLargeExp = -1;
+   if (pBucket->nSize == 0) nLargeExp = HeapLargeClass (&nSize);
+
+   THeapBlockHeader *pBlockHeader;
+   if (pBucket->nSize > 0 && (pBlockHeader = pBucket->pFreeList) != 0) {
+       pBucket->pFreeList = pBlockHeader->pNext;
+       m_nFreeListBytes -= pBlockHeader->nSize; // Onyx
+   } else if (nLargeExp >= 0 && (pBlockHeader = m_pLargeFreeList[nLargeExp]) != 0) {
+       m_pLargeFreeList[nLargeExp] = pBlockHeader->pNext;
+       m_nFreeListBytes -= pBlockHeader->nSize; // Onyx
+   } else { ... bump m_pNext ... }

```

```

@@ -226,6 +281,7 @@ void CHeapAllocator::DoFree (void *pBlock)    // (bucket free
path)
    pBucket->pFreeList = pBlockHeader;
+    m_nFreeListBytes += pBlockHeader->nSize;    // Onyx
@@ -237,6 +293,19 @@ void CHeapAllocator::DoFree (void *pBlock)
+    // Onyx: no matching bucket -> power-of-two "large" block: return it to its size
+    // class free list so it is reused on the next same-class allocation (no leak).
+    int nLargeExp = HeapLargeIndex (pBlockHeader->nSize);
+    if (nLargeExp >= 0) {
+        m_SpinLock.Acquire ();
+        pBlockHeader->pNext = m_pLargeFreeList[nLargeExp];
+        m_pLargeFreeList[nLargeExp] = pBlockHeader;
+        m_nFreeListBytes += pBlockHeader->nSize;
+        m_SpinLock.Release ();
+        return;
+    }

--- a/include/circle/sysconfig.h
+++ b/include/circle/sysconfig.h
@@ -85,6 +85,10 @@
#ifndef HEAP_BLOCK_BUCKET_SIZES
// Small fixed buckets for common sizes. Larger requests (e.g. window canvases) no
// longer need a matching bucket: Onyx's heap allocator rounds anything above the
top
// bucket up to a power-of-two class and reuses it from a per-power free list (see
// HEAP_LARGE_LISTS / heapallocator.cpp), so they are no longer lost on free.
#define HEAP_BLOCK_BUCKET_SIZES    0x40,0x400,0x1000,0x4000,0x10000,0x40000,0x80000
#endif

```

3. Free-space accounting (for the `meminfo` kapi)

Why. The `meminfo` kapi (and the memory monitor) reports total / free / app memory. The "free" figure must include both the unallocated region *and* the freed blocks/pages sitting on the allocators' free lists (reusable). Circle exposed only the unallocated region, so we add free-list accessors. `memory.h` is header-only; the `pageallocator` counter needs a `libcircle` rebuild.

```

--- a/include/circle/memory.h
+++ b/include/circle/memory.h
@@ -155,6 +155,25 @@ public:
    static void PageFree (void *pPage) { s_pThis->m_Pager.Free (pPage); }

+    // Onyx: free space (bytes) of the page allocator region not yet handed out.
+    static size_t GetPagerFreeSpace (void) { return s_pThis->m_Pager.GetFreeSpace
+(); }
+    // Onyx: + freed pages on the pager free list (reusable).
+    static size_t GetPagerFreeListSpace (void) { return s_pThis->m_Pager.GetFreeListSpace
+(); }
+
+    // Onyx: freed heap blocks on the bucket/large free lists (reusable). Add to
+    // GetHeapFreeSpace(HEAP_ANY) for the true free heap.
+    static size_t GetHeapFreeListSpace (void) const
+    {
+    +#if RASPPPI >= 4
+        return s_pThis->m_HeapLow.GetFreeListSpace () + s_pThis->m_HeapHigh.GetFreeListSpace
+();
+    +#else
+        return s_pThis->m_HeapLow.GetFreeListSpace ();
+    +#endif
+    }

```

```

+
+ static void DumpStatus (void)
+
+ --- a/include/circle/pageallocator.h
+++ b/include/circle/pageallocator.h
@@ -48,6 +48,10 @@ public:
+ size_t GetFreeSpace (void) const;
+
+ /// \return Onyx: bytes of freed pages currently on the free list (reusable). Add
+ /// to GetFreeSpace() for the true total free space.
+ size_t GetFreeListSpace (void) const;
+
+ void *Allocate (void);
@@ -67,6 +71,7 @@ private:
+ TFreePage *m_pFreeList;
+ unsigned m_nFreeListCount; // Onyx: pages currently on m_pFreeList
+ CSpinLock m_SpinLock;
+
+ --- a/lib/pageallocator.cpp
+++ b/lib/pageallocator.cpp
@@ -30,7 +30,8 @@ CPageAllocator::CPageAllocator (void)
- m_pFreeList (0)
+ m_pFreeList (0),
+ m_nFreeListCount (0)
{
}
@@ -49,6 +50,11 @@ size_t CPageAllocator::GetFreeSpace (void) const
+size_t CPageAllocator::GetFreeListSpace (void) const // Onyx: reusable freed pages
+{
+ return (size_t) m_nFreeListCount * PAGE_SIZE;
+}
@@ -68,6 +74,7 @@ void *CPageAllocator::Allocate (void)
+ m_pFreeList = pFreePage->pNext;
+ m_nFreeListCount--; // Onyx: page taken off the free list
@@ -103,6 +110,7 @@ void CPageAllocator::Free (void *pPage)
+ m_pFreeList = pFreePage;
+ m_nFreeListCount++; // Onyx: page returned to the free list

```

4. Multi-core enabled (ARM_ALLOW_MULTI_CORE)

include/circle/sysconfig.h: the define is un-commented, so Circle builds its multi-core support (CMultiCoreSupport, real spin locks, per-core stacks and MMU setup, IPIs). The Onyx kernel starts cores 1-3 at boot; core 1 is the **sound producer** (it renders the audio chunks ahead, see [Kernel Internals §12](#)); the scheduler, the processes and the interrupts stay on core 0. The memory map is unchanged (Circle always reserves the stacks of the 4 cores).

```

-// #define ARM_ALLOW_MULTI_CORE
+ #define ARM_ALLOW_MULTI_CORE // Onyx: core 1 runs the sound producer
(kernel sys/sound.cpp)

```

Changing this define changes code in **every** Circle library: after updating the submodule, run `make clean` in each library before rebuilding them (the `.o` files do not depend on `sysconfig.h`) — see the [Developer Guide §2](#).

5. Shift + navigation keys

Circle's keymap only has **Ctrl** variants of the navigation keys (`KeyCtrlUp ... → ESC[1;5A ...`); Shift + an arrow produced nothing. The patch adds eight special keys **at the end of**

`TSpecialKey` (just before `KeyMaxCode`), so every existing code — the values stored in the `.kmap` layout files — keeps its number:

Key	String
KeyShiftHome / KeyShiftEnd	ESC[1;2H / ESC[1;2F
KeyShiftPageUp / KeyShiftPageDown	ESC[5;2~ / ESC[6;2~
KeyShiftUp / Down / Left / Right	ESC[1;2A / B / D / C

The Onyx layouts (`tools/keymaps/maps/*.h` → `genkeymaps.py` → `SD:/etc/keymaps/*.kmap`) put them in the **Shift** column of the physical keys 0x4A-0x52. The kernel's key parser (`gui/window.cpp`, `NextKey`) turns any `ESC[n;mX` into the plain `KEY_*` code; apps read the modifier with `kapi_get_modifiers` (text selection in `gtk::Textarea` / `RichTextBox`).

6. Partial display update

`C2DGraphics::UpdateDisplay ()` sends the whole off-screen buffer to the frame buffer. The Onyx compositor redraws only the damaged rectangles of the screen, so the patch adds an overload that sends one rectangle:

```
void UpdateDisplay (unsigned nPosX, unsigned nPosY, unsigned nWidth, unsigned nHeight);
```

It clips the rectangle to the screen, copies its rows into one contiguous block (a screen-sized buffer allocated on first use: cached memory, fast) and hands it to `CBcmFrameBuffer::SetArea` — the frame buffer's own 2D DMA copy with the destination pitch, as the full update uses. With `VSync` (page flipping) or a display that is not the frame buffer it falls back to the full `UpdateDisplay ()`.

7. Yielding SD waits (FatFs lock + EMMC wait hooks)

The EMMC driver waits for the card in a busy loop (`CEMMCDevice::TimeoutWait`) unless `NO_BUSY_WAIT` is set, and Onyx's kernel is not preempted: every SD command kept the CPU, and a directory walk or a big read froze the GUI and the network 100–200 ms at a time. `NO_BUSY_WAIT` is global (USB, sound, every `CGenericLock`), so the patch adds two **weak** hooks instead; without a definition, Circle behaves exactly as upstream:

- `ffsystem.cpp`: `ff_mutex_take` / `ff_mutex_give` call `OnyxFsLockTake (vol) / OnyxFsLockGive (vol)` when they are defined, instead of the `CGenericLock` (a spin lock).
- `emmc.cpp`: once a `TimeoutWait` has spun 2 ms (the normal latency of a command; yielding earlier cost another task's whole time slice per command: reads fell to 2 MB/s), each turn of its loop calls `OnyxDriverWait ()` when it is defined.

The kernel defines them in `kernel/sys/fslock.cpp`: a sleeping, re-entrant volume lock (waits `Yield`), and a wait hook that yields when IRQs are on. See [docs/02](#) (preemption).

7b (2026-10-01): many short waits. One command can now carry a whole file's contiguous clusters (patch 10): a long write waits for the card a few hundred μ s per block, every wait under the 2 ms spin, and kept core 0 for 200 ms (`stall: jet:cache ran 213 ms without yielding`, the PC in `TimeoutWait`). `emmc.cpp` now also calls a second weak hook, `OnyxDriverPoll ()`, at each turn of a wait shorter than 2 ms. The kernel's (`fslock.cpp`) yields once the task has run 10 ms since its last `Yield` (the scheduler notes that time), with the same conditions as `OnyxDriverWait` (a scheduler on this core, IRQs on, not core 1's crash dump). Safe for the same reasons as patch 7: the volume lock is held (no other task sends the card a command — and nothing outside FatFs drives the card while tasks run), and the SD host holds a PIO transfer between two blocks until it is served (upstream's `NO_BUSY_WAIT`

yields at every turn of these waits). Reads (40 MB/s) yield about every 400 KB, writes every 50–100 KB. Without the hook, Circle behaves as upstream.

8. Sector cache in the FatFs disk layer

FatFs reads the FAT and the directories one sector at a time (through its window), and each `disk_read` is a whole SD command round trip (plus a CMD13 status check in the EMMC driver): opening a file deep in a big folder, or listing it, asked the card again for every sector, every time. `diskio.cpp` now keeps the **single-sector reads** in a direct-mapped cache (2048 slots x 512 bytes = 1 MB, allocated on first use; key = sector and drive).

Write-through: `disk_write` writes the card first, then updates the cached copies of the sectors written (or drops them if the write failed), so the cache never holds anything the card does not. The big multi-sector reads of file data go straight to the device.

`disk_initialize` (a mount) and the device's removal forget the drive's sectors.

`disk_cache_enable (0/1) / disk_cache_stats (&hits, &misses)` (declared in `diskio.h`); Onyx turns it off with `sdcache=0` in `cmdline.txt`.

Also: the bounce buffer for unaligned transfers is now **one per volume** (the SD driver can yield in the middle of a transfer since patch 7, so two volumes' transfers can overlap; FatFs locks per volume).

Test on the PC: `tools/tests/run_fs_test.sh` builds the fork's `ff.c` + `diskio.cpp` against stub Circle headers, formats a 96 MB RAM disk (FAT32) and runs 4000 random operations (create, overwrite, append, read back, delete, rename, list, remount) checked against a model, with the cache on and off: both pass and leave the same image byte for byte; the cache cuts the device reads about 3.4 times.

9. High Speed SD at run time

Upstream enables the SD High Speed mode (50 MHz, SD 1.1+ cards: CMD6 switch) only with the compile option `SD_HIGH_SPEED`, off by default, and then only raises the clock. The patch:

- `static void CEMMCDevice::SetHighSpeed (boolean)` (before `Initialize`) chooses it at run time (`SD_HIGH_SPEED` still forces the upstream behaviour); `IsHighSpeed ()` tells whether the card switched;
- before raising the clock, it sets the host's **High Speed Enable** bit (`CONTROL0` bit 2, `HCTL_HS_EN` of the SD Host Controller spec), so the controller samples with the High Speed timing;
- the 64-byte CMD6 status buffer is word aligned (the PIO transfer asserts it).

Onyx turns it on by default (`sdhs=0` in `cmdline.txt` turns it off, for a card that misbehaves).

10. Multi-cluster transfers in FatFs

`f_read` / `f_write` move whole sectors straight between the caller's buffer and the disk, but upstream **clips each transfer at the cluster boundary**: with small clusters (a card formatted with 512-byte clusters) every SD command carried 512 bytes, and the commands' latency (~240 µs each on the Pi 4) capped reads at 2 MB/s (`fsbench`: 1 MB = 2064 commands; the data port itself moved 1 MB in 23 ms). The patch goes on across the **next clusters while they follow on the disk** (`get_fat (cl) == cl + 1` for a read; `create_chain` — follow or allocate — for a write), up to the size asked, then sets `fp->clust` to the cluster of the last sector moved, so the loop resumes exactly as before. A cluster allocated ahead that does not follow simply stays in the chain (the next turn finds it, as it would have allocated it). Not with FastSeek (`cltbl`). The allocation order is unchanged:

`run_fs_test.sh` gets the **same disk image as upstream's `ff.c`**, byte for byte, with 14 x fewer reads and 6 x fewer writes on 512-byte clusters.

11. FatFs fast seek

`FF_USE_FASTSEEK` is **1**: `f_lseek` can use a file's **cluster map** (`FIL.cltbl`) instead of walking the FAT chain from the start at each backward seek. The kernel's `kapi_seek` (ABI v57) builds it at the first seek of a file bigger than 4 MB (`f_lseek (fp, CREATE_LINKMAP)`, the table grown once if the file is fragmented) and `kapi_close` frees it: a GameCube disc image (1.4 GB) is then read anywhere at once. The option adds `cltbl` to `FIL`: **the kernel must be rebuilt clean** with the new `libfatfs`.

12. 2D DMA with a source stride, asynchronous partial updates

Why. Circle's `SetupMemCopy2D` only knows a destination stride: the source must be one contiguous block. So patch 6 gathered each damaged rectangle's rows into a buffer with the CPU (a copy as big as the DMA's) before the frame buffer's DMA copied it to the screen, and `SetArea` then **busy-waited** for the DMA: core 0 (the GUI, the apps' tasks) stood still for the whole transfer (2–4 ms for a full 1080p screen).

What. Both DMA engines have a source stride too (legacy `STRIDE` bits 0–15, DMA4 `SRCI` bits 16–31):

```
void SetupMemCopy2D (void *pDestination, const void *pSource,
                    size_t nBlockLength, unsigned nBlockCount, size_t nBlockStride,
                    unsigned nBurstLength, size_t nSourceStride); // CDMAChannel,
CDMA4Channel
void CBcmFrameBuffer::SetAreaPitch (const TArea &, const void *pPixels, unsigned
nSourcePitch,
                                   TAreaCompletionRoutine *pRoutine = nullptr, void
*pParam = nullptr);
void C2DGraphics::UpdateDisplayAsync (unsigned x, unsigned y, unsigned w, unsigned h,
// w = 0: all
                                   CDisplay::TAreaCompletionRoutine *pRoutine,
void *pParam);
```

The overloads clean only the rectangle's rows from the data cache (not the whole span). The partial `UpdateDisplay (x, y, w, h)` of patch 6 now uses `SetAreaPitch` (the gathering buffer is gone). `UpdateDisplayAsync` starts the DMA and returns; the routine runs from the DMA's interrupt.

Polled, without the interrupt. Under a heavy GPU load (Ocarina of Time in `n64emu`) a completion interrupt was now and then never seen, and the compositor waited for ever (the screen, the apps presenting, the sound feeder behind them: all stuck). So the compositor (`CCompositorTask::Present`, `kernel.cpp`) no longer uses `UpdateDisplayAsync`; it starts the DMA and polls it between yields:

```
boolean C2DGraphics::UpdateDisplayStart (unsigned x, unsigned y, unsigned w, unsigned
h); // FALSE: done at once
boolean C2DGraphics::UpdateDisplayPoll (void); // TRUE once the
transfer is over
boolean CBcmFrameBuffer::SetAreaPitchStart (const TArea &, const void *, unsigned
nSourcePitch);
boolean CBcmFrameBuffer::SetAreaPoll (void); // (frees the frame
buffer's DMA channel)
boolean CDMAChannel::Poll (void); boolean CDMA4Channel::Poll (void); // not active
any more
```

It draws nothing into the off-screen buffer until the poll says done. `cmdline.txt dispdma=0` still makes every copy synchronous (`UpdateDisplay`).

13. SD card partitions as volumes, exFAT

Why. Upstream maps each FatFs volume to a whole device (`emmc1`, `umsd1...`): only the first FAT partition of the card was seen, and exFAT was off (files ≤ 4 GB, no exFAT user partition). The Pi 4 boots only from a FAT32 partition 1; a second partition, formatted exFAT, can hold the big files (ROMs, disc images).

What.

- `ffconf.h`: `FF_VOLUMES 9`, `FF_VOLUME_STRS` `"SD", "SD1", "SD2", "SD3", "USB", "USB2", "USB3", "FD", "NVME"`; `FF_FS_EXFAT 1` (needs `FF_USE_LFN`; `FSIZE_t` becomes 64-bit → rebuild the kernel clean).
- `ffconf.h`: `FF_MULTI_PARTITION 1`. `diskio.cpp`: the physical drives are `emmc1` (the whole card, as upstream), `umsd1...umsd3`, `ufd1`, `nvme1`; **VolToPart** maps `SD:` to the card's partition 0 (*auto*: FatFs takes the first FAT volume, exactly as before the change) and `SD1:...SD3:` to MBR partitions 2–4 of the same drive (their type is not checked: FAT or exFAT is recognised from the boot sector). `disk_initialize` keeps a drive already initialised (a second partition of it) instead of forgetting its sector cache. (A first version opened Circle's partition devices `emmc1-1...`: dropped, `SD:` must be found exactly as before.)
- `ff.c`: the multi-cluster **write** extension (patch 10) is skipped on exFAT: a contiguous exFAT file (`NoFatChain`) that becomes fragmented must first have its FAT chain written, which upstream does cluster by cluster (reads keep the fast path).

Rebuild everything that includes `ff.h` after changing `ffconf.h`: `FF_FS_EXFAT` makes `FSIZE_t` 64-bit, so `FIL` / `FATFS` / `FILINFO` grow. Besides `libfatfs` and the kernel (clean), that is `addon/wlan` (`libwlan.a`: the Wi-Fi firmware is read through FatFs) and `addon/wlan/hostap/wpa_supplicant` (make `-f Makefile.circle`: it reads `wpa_supplicant.conf`). With stale ones, their `FIL` on the stack is too small: FatFs writes past it on the network core — Onyx booted but the Wi-Fi never came up.

The kernel mounts `SD1:...SD3:` when `f_mount` succeeds and shares one lock slot between the four SD volumes (one card). `tools/tests/run_fs_test.sh` runs the FatFs test on FAT32 **and exFAT** images (the fork's image must equal upstream's), and first `fstest` parts: an MBR card image with partition 1 FAT32 and partition 2 exFAT, `SD:` and `SD1:` mounted, a file on each, `SD2:` absent.

14. Wi-Fi: a DHCP restart (join another network while running)

Why. `CWPASupplicant` reads `wpa_supplicant.conf` once, at boot, and once bound, Circle's DHCP client sleeps until its lease's renewal time: joined to another network, the Pi would keep the old address.

What. `CDHCPClient::Restart ()` (static flag `s_bRestart`, `include/circle/net/dhcpclient.h`, `lib/net/dhcpclient.cpp`): the bound state checks it every 0.5 s (it slept 10 s at a time) and, if set, halts the network and starts over (discover / request).

The configuration itself is read again **without touching hostap** (a submodule of upstream's): `wpa_supplicant` registers its `SIGHUP` handler (`wpa_supplicant_reconfig`: every interface reloads its configuration, deauthenticates and joins the highest `priority` network in range) with `eloop_register_signal_reconfig`, which Circle's `eloop` ignores. The Onyx kernel is linked with `--wrap=eloop_register_signal_reconfig` (`kernel/Makefile`): it keeps

that handler, and `kapi_wlan_reconnect` (ABI v60) runs it from the supplicant's own event loop (a 0 s eloop timeout), then asks `CDHCPClient::Restart ()`.

Not a patch: build configuration

One required setting is a **configure option**, not a source change: Onyx renders 32-bit pixels, so Circle must be configured with **DEPTH=32**:

```
cd circle && ./configure -r 4 -p aarch64-none-elf- -d DEPTH=32 -f
```

See the [Developer Guide §2](#) for the full Circle build.

15. A crash area kept out of the heap

Why. A frozen Pi (a kernel hang, a bus wedged by the GPU) leaves nothing to read: `kmsg` is gone with the session. Onyx keeps a crash record in RAM that survives the watchdog's reboot.

What. `CMemorySystem::SetupHighMemAbove4G` (`lib/memory64.cpp`) keeps the top `ONYX_CRASH_AREA_SIZE` (64 KB) of the first RAM segment it adds above 3 GB (the [3 GB, RAM top) low-RAM top from the device tree, or the [4 GB, RAM end) chunk of the fallback — the case of an 8 GB Pi 4 without a captured device tree) out of the high heap and publishes its address in `u64 g_ulOnyxCrashArea` (`include/circle/memory.h`; 0 when the board has no RAM there, e.g. 2 GB).

`CSpinLock::TryAcquire ()` (`include/circle/spinlock.h`, `lib/spinlock.cpp`): one try, FALSE (nothing held, the IRQ level restored) when another core holds the lock — core 1's crash watch never waits for ever behind core 0. It stays mapped NORMAL (cacheable): the kernel cleans each write to the point of coherency. See [docs/02 §13](#).

16. TCP: a peer gone before it was accepted

Why. A client that connects and closes at once (a port scan, a script checking a port) left the listening connection woken without a foreign address: `CTCPConnection::Accept` then called `CIPAddress::Set` on it, whose assertion (`rAddress.m_bValid`) is a panic — every core halted, the Pi restarted (seen with `telnetd`: `lastcrash.txt` showed it, in a restart loop while a monitor on the PC probed port 23).

What. `lib/net/tcpconnection.cpp`, `Accept`: after the wait, an error (`m_nErrno`) or a connection without a foreign address returns `-NET_ERROR_CONNECTION_RESET`; `CSocket::Accept` then returns 0 (that peer not accepted) and listens again. The same bug is in upstream Circle.

17. The host name set after the constructor

Why. The name DHCP announces is given to `CNetSubSystem`'s constructor, and Onyx builds it as a member of `CKernel` — before the SD card is mounted, so before `SD:/etc/system.ini` (its `hostname=`, written by Setup, the first-run wizard) can be read.

What. `include/circle/net/netsubsystem.h`: `void SetHostname (const char *pHostname)` (inline: `m_Hostname` replaced), called by the kernel before the network's bring-up task starts (`Initialize` makes the DHCP client with the name). Header only: no library to rebuild.

18. DNS: the answer polled, not a second slept

Why. `CDNSClient::Resolve` sent its query, slept 1000 ms, then read the answer once: every name lookup cost a second even when the answer came in 5 ms (a web page asks for several hosts: NetSurf spent seconds in DNS alone, its connects one after another).

What. `lib/net/dnsclient.cpp`: after each send, the socket is read every 5 ms (`MSG_DONTWAIT`) for up to 1000 ms; the three tries are kept. (The kernel also caches the answers: `kernel/sys/net.cpp`, docs/02.)

19. TCP: a 64 KB receive window

Why. The advertised receive window was 10 segments (14600 bytes), with no window scaling: a connection could not carry more than 14.6 KB a round trip (about 730 KB/s at 20 ms, 290 KB/s at 50 ms) -- well under the Wi-Fi's rate.

What. `lib/net/tcpconnection.cpp`: `TCP_CONFIG_WINDOW` is 44 segments (64240 bytes, the most a 16-bit window field holds); the reassembly queue follows it. Rebuild `lib/net`.

20. TCP: real duplicate ACKs only

Why. `CTCPConnection::PacketReceived` counted **every** segment whose ACK did not advance as a duplicate ACK — the peer's own data segments too, and ACKs with nothing in flight, window updates — and the count was reset only by an advancing ACK. The remote desktop client (Onyx Remote) sends small messages (READY, the pointer's moves) while the Pi streams the screen: three in a row between two ACKs started a **spurious fast retransmit + fast recovery** (cwnd cut to about two segments), and the Pi's sending crawled while the mouse moved.

What. `lib/net/tcpconnection.cpp`: in the "ACK not advancing" branch, `OnDuplicateAck` is called only for a duplicate ACK as RFC 5681 §2 defines it: no data and no SYN / FIN (`nSEG_LEN == 0`), the ACK number `SND.UNA`, the advertised window unchanged (`m_nSND_WND`), and data outstanding (`FLIGHT_SIZE > 0`). Other segments neither count nor reset the count. Fast retransmit and fast recovery (NewReno) are unchanged otherwise. The same bug is in upstream.

21. TCP: the retransmission timeout

Why. `retranstimeoutcalc.cpp` started at 3 s (RFC 1122's value) and never went below 1 s (RFC 6298's conservative minimum): on a LAN or Wi-Fi peer a few ms away, a segment lost at the end of a burst (nothing behind it to cause duplicate ACKs) stalled the stream a whole second — the remote desktop froze.

What.

- `INITIAL_RTO` 1 s (RFC 6298 2.1); `MIN_RTO` stays **1 s**. A 200 ms minimum (Linux's `TCP_RTO_MIN`) was tried on 2026-10-01 and undone the same day: the remote desktop's throughput over Wi-Fi to a Windows PC collapsed (64 KB sends taking 0.5–2 s instead of a few ms, ~50x slower). Windows delays its ACKs up to 200 ms and Wi-Fi adds bursts of delay: the timer fired spuriously and each timeout cut the congestion window to one segment; Linux survives that floor with F-RTO / undo of spurious timeouts, which Circle lacks. RFC 6298's granularity term G is 200 ms (was 1 tick).
- **Karn's algorithm** also after a fast retransmit: `CRetransmissionTimeoutCalculator::SegmentResent` (new) marks a segment sent again by `ResendSegment` (fast retransmit, a partial ACK in fast recovery), so its ACK

gives no RTT sample (only a timeout marked it before). A valid sample resets the backed-off RTO, as before.

- `MAX_RETRANSMISSIONS` stays 5 (63 s from 1 s), `MAX_SYN_RETRANSMISSIONS` 5 (63 s from the 1 s initial RTO; it was 189 s from 3 s).
- After a SYN retransmitted, RFC 6298 (5.7) wants the data phase to start at 3 s: the RTO is then the backed-off one (≥ 2 s) until the first sample — close enough, left as is.

22. `CSocket::Send`: the bytes queued

Why. `CSocket::Send` cuts a buffer into MSS-sized chunks and queues them one by one; when the connection's queue stays full past the send timeout (Onyx sets 5 s), a chunk fails — and `Send` answered the **error**, although the chunks before it were queued and would be sent. The kernel's `NetTcpSend` (32 KB requests) then counted none of that request, and an app sending "the rest" again put those bytes **twice** in the stream (the remote desktop's stream broke: its client read garbage).

What. `lib/net/socket.cpp`: `Send` (and `SendTo`, for TCP) answers the bytes queued when a later chunk fails, if any (as POSIX `send`); the error comes back at the next call. The kernel's `NetTcpSend` adds its requests' counts and stops at a short one.

Test (patches 20–22): `tools/tests/run_circlenet_test.sh` builds the fork's real `tcpconnection.cpp`, `retranstimeoutcalc.cpp`, `socket.cpp` (+ the buffers, queues, checksum) on the PC against stub Circle headers (`tools/tests/circlenet/stub`: a simulated clock and kernel timers, no tasks, a network layer that hands each segment to the test) and plays the peer: the peer's data segments and window updates are no duplicate ACKs, three real ones still start a fast retransmit and a full ACK ends the recovery, the RTO (200 ms, backed off, reset by a sample, 1 s for a SYN, Karn after a fast retransmit, a dead peer given up after 51 s), and `Send`'s count after a timeout. Against the unpatched sources it fails 15 of its 25 checks.

Contributions to upstream Circle

The fork's changes useful to every Circle user are prepared as clean pull-request branches on upstream `develop` (the scheduler's `WakeTasks` race, the SD High Speed fixes, large heap blocks reuse, partial display updates with a DMA source stride, the DHCP restart), with the patches and the pull-request texts: see `docs/circle-upstream/README.md`.

Updating the fork (submodule)

```
# rebase the Onyx patches onto a newer upstream, inside the submodule:
git -C circle fetch upstream
git -C circle rebase upstream/master onyx      # resolve conflicts in the patched
files
# rebuild the affected libraries: libcircle (heap + page accounting), libinput
(keymap)
# then record the new fork commit in the superproject:
git add circle && git commit -m "Bump circle submodule"
```